NAME:- KISHORI KAMALI M

REG NO:23MIC0055

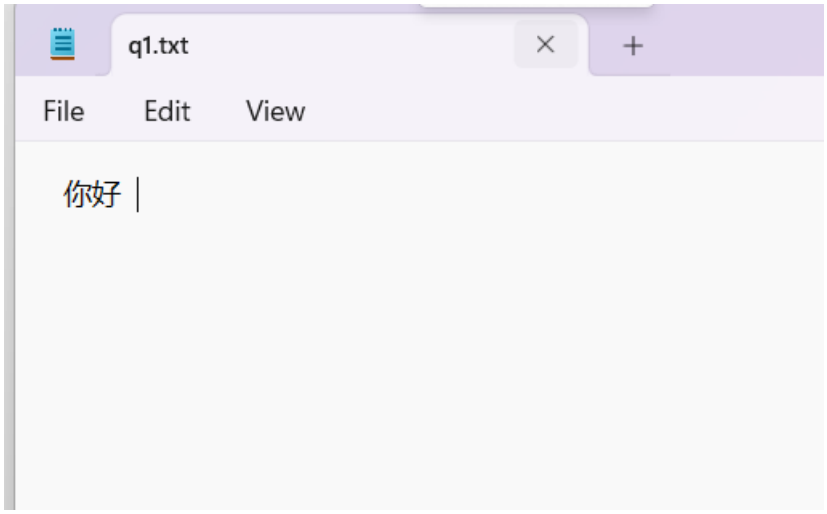
SUBJECT:- ADVANCED PYTHON PROGRAMMING(L31+L32+L49+L50)

LAB ASSESSMENT 1

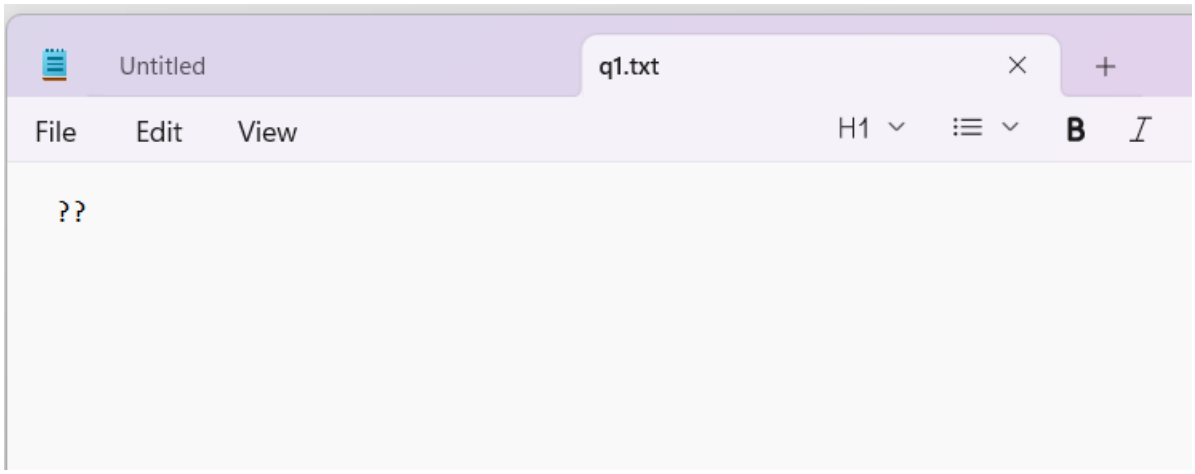
QUESTION1:-

UNICODE CHINESE CHARACTER USING ANSI

Before saving



After reopening it



QUESTION 2:-

STUDENT ARE ASKED TO DO 4 OPERATIONS/FUNCTIONS TO DO WITH 2 PYTHON LIBARARY WITH TWO DIFFERENT FORMAT

I HAVE TAKEN PDF,PPT AS TWO FORMATS

1.PYPDF-(PYTHON LIBARARY)

- ROTATE THE PDF AND SAVE IT AS ANOTHER PDF

```
[ ]: #23MIC0055 KISHORI KAMALI M
     #QUESTION 1 NEED TO DO 4 FUNCTION/OPERARTIONS USING PYPDF WITH PDF FORMAT
```

```
[2]: pip install pypdf
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting pypdf
  Downloading pypdf-6.14.2-py3-none-any.whl.metadata (7.2 kB)
Downloading pypdf-6.14.2-py3-none-any.whl (349 kB)
Installing collected packages: pypdf
Successfully installed pypdf-6.14.2
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
*[4]: from pypdf import PdfReader, PdfWriter

reader = PdfReader("C:\\Users\\KISHORI KAMALI\\Downloads\\PDF1.pdf")
writer = PdfWriter()

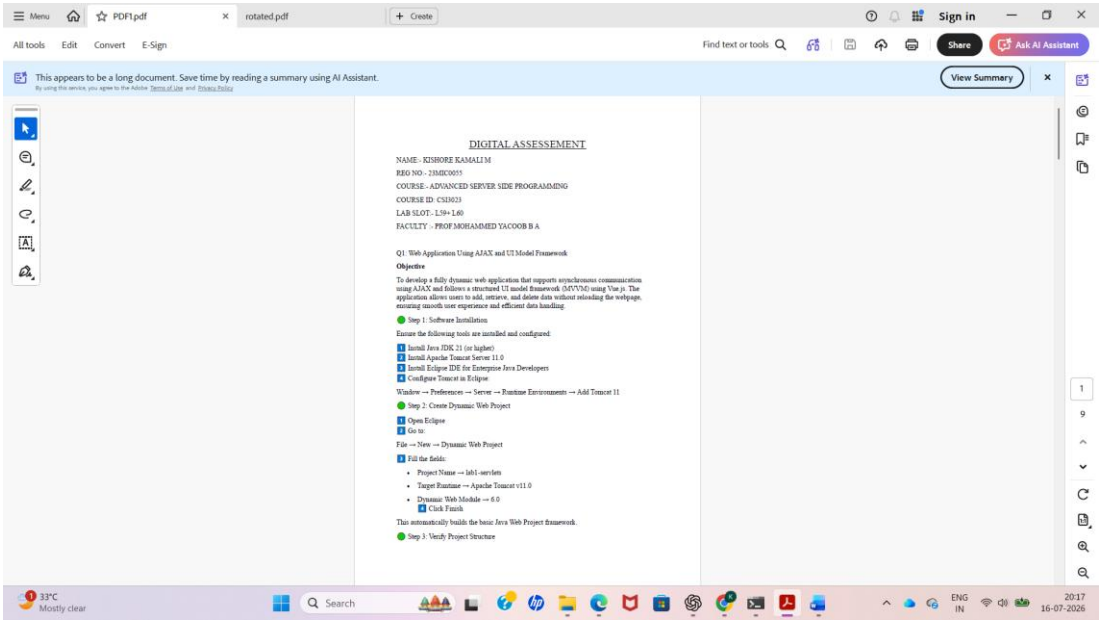
for page in reader.pages:
    page.rotate(90)      # ROTATE 90 DEGREE
    writer.add_page(page)

with open("rotated.pdf", "wb") as f:#SAVING THE ROATED PDF AS rotated.pdf
    writer.write(f)

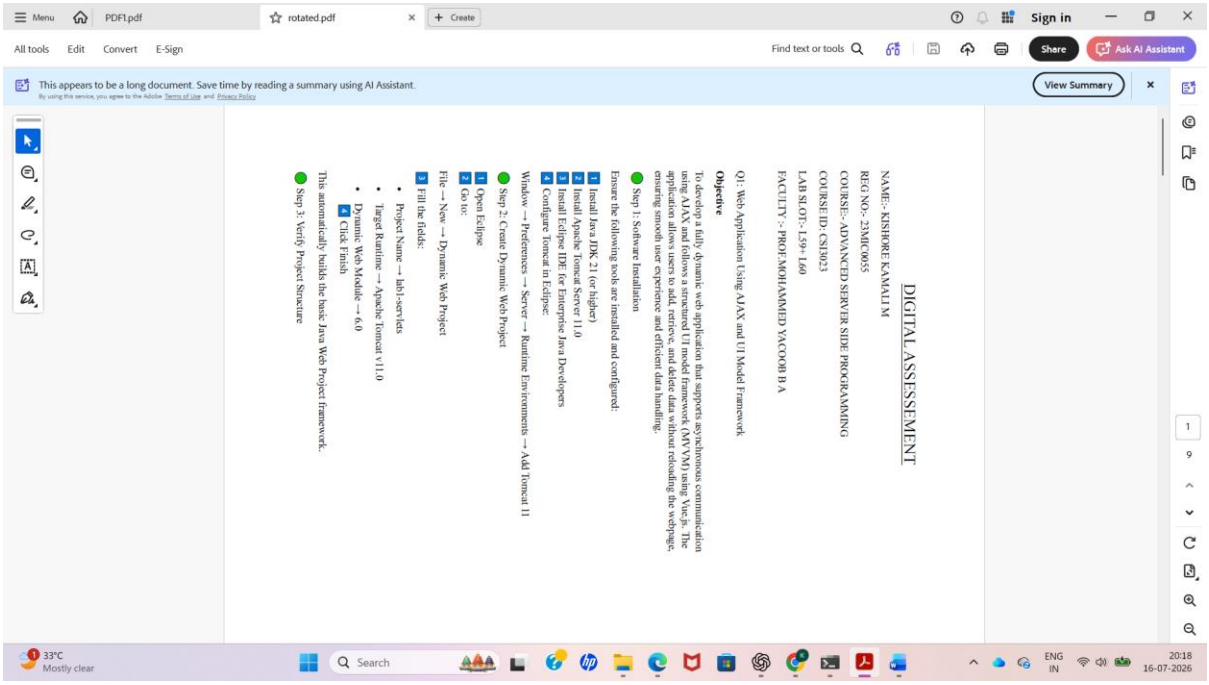
print("PDF rotated successfully.")# after successfully rotated and saved as another pdf i am printing th success message
```

OUTPUT:-

BEFORE ROTATING :-



AFTER ROTATED AND SAVED AS “rotate.pdf”



- EXTRACTING TEXT :-

```
]: from pypdf import PdfReader

reader = PdfReader("C:\\Users\\KISHORI KAMALI\\Downloads\\PDF1.pdf")

for i, page in enumerate(reader.pages):
    print(f"\n----- Page {i+1} -----")
    text = page.extract_text()#extracting the text using this function and store it in the variable called text

    print(text)#printing the variable which has the extracted text
```

OUTPUT:-

```
----- Page 1 -----
DIGITAL ASSESSEMENT
NAME:- KISHORE KAMALI M
REG NO:- 23MIC0055
COURSE:- ADV ANCED SERVER SIDE PROGRAMMING
COURSE ID: CSI3023
LAB SLOT:- L59+ L60
FACULTY :- PROF.MOHAMMED YACOOB B A

Q1: Web Application Using AJAX and UI Model Framework
Objective
To develop a fully dynamic web application that supports asynchronous communication
using AJAX and follows a structured UI model framework (MVVM) using Vue.js. The
application allows users to add, retrieve, and delete data without reloading the webpage,
ensuring smooth user experience and efficient data handling.
   Step 1: Software Installation
Ensure the following tools are installed and configured:
```

- EXTRACTING THE IMAGES FROM THE PDF AND SAVING IT SEPARATELY WITH THE UNIQUE NAME

```
[6]: from pypdf import PdfReader

reader = PdfReader("C:\\Users\\KISHORI KAMALI\\Downloads\\PDF1.pdf")

image_no = 1

for page_num, page in enumerate(reader.pages):
    for image in page.images:
        with open(f"page{page_num+1}_image{image_no}.png", "wb") as f:#extracting the images and saving it separately
            f.write(image.data)
        print(f"Saved: page{page_num+1}_image{image_no}.png")# printing the saved images names
        image_no += 1
```

OUTPUT:-

```
        image_no += 1

Saved: page5_image1.png
Saved: page5_image2.png
Saved: page6_image3.png
Saved: page7_image4.png
```

```
[ ]:
```



- READING THE METADATA AND PRINTING IT

```
from pypdf import PdfReader

# Open the PDF
reader = PdfReader("C:\\Users\\KISHORI KAMALI\\Downloads\\PDF1.pdf")

# Get metadata
metadata = reader.metadata#READING THE METADATA AND STORE IT IN THE VARIABLE CALLED MEATDATA

print("-------- PDF Metadata --------")

if metadata:#PRINTIGN THE DETAILS STORED IN THE VARIABLE CALLED METADATA
    print("Title      :", metadata.title)
    print("Author     :", metadata.author)
    print("Subject    :", metadata.subject)
    print("Creator    :", metadata.creator)
    print("Producer   :", metadata.producer)
    print("Keywords   :", metadata.keywords)
    print("Creation Date :", metadata.creation_date)
    print("Modification Date :", metadata.modification_date)
else:
    print("No metadata found in the PDF.")
```

OUTPUT:-

```
    print("No metadata found in the PDF.")

-------- PDF Metadata --------
Title      : None
Author     : KISHORI KAMALI
Subject    : None
Creator    : Microsoft Word
Producer   : None
Keywords   : None
Creation Date : 2025-12-19 17:13:54+00:00
Modification Date : 2025-12-19 17:13:54+00:00
```

2. I HAVE TAKEN PPT AS SECOND FORMAT AND I AM USING python-pptx (PYTHON LIBARARY)

- MANIPULATING THE PPT TEXT BY CHANGING THE COLOR STYLE

```
from pptx import Presentation
from pptx.util import Pt
from pptx.dml.color import RGBColor

prs = Presentation(r"C:\Users\KISHORI KAMALI\Downloads\PPT1.pptx")

for slide in prs.slides:
    for shape in slide.shapes:

        if shape.has_text_frame:

            for paragraph in shape.text_frame.paragraphs:

                for run in paragraph.runs:#MANIPULATING THE PPT TEXT BY CHNAGING THE STYLE

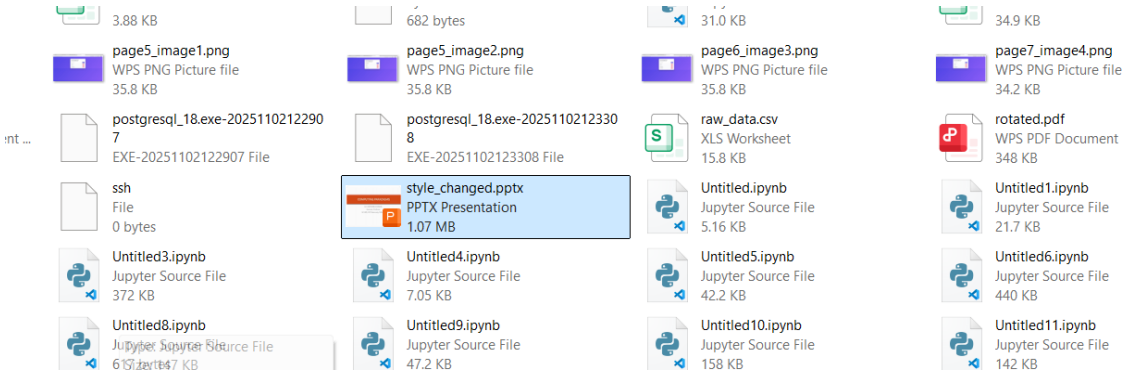
                    run.font.name = "Arial"
                    run.font.size = Pt(24)
                    run.font.bold = True
                    run.font.italic = False
                    run.font.color.rgb = RGBColor(255, 0, 0)

prs.save("style_changed.pptx")#SAVE IT AS ANOTHER PPT CALLED style_changed.pptx

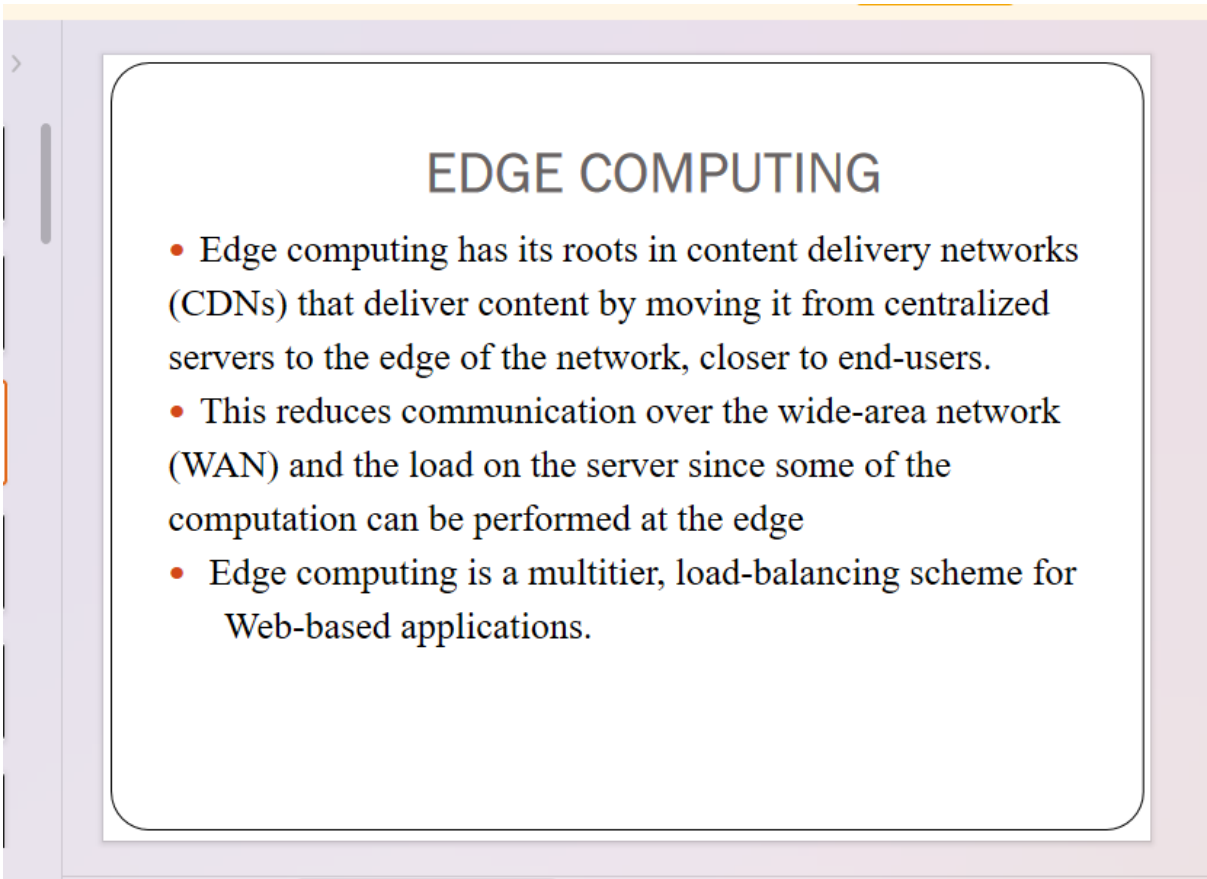
print("Style changed successfully.")#DISPLAYING THE SUCCESS MESSAGE
```

OUTPUT:-

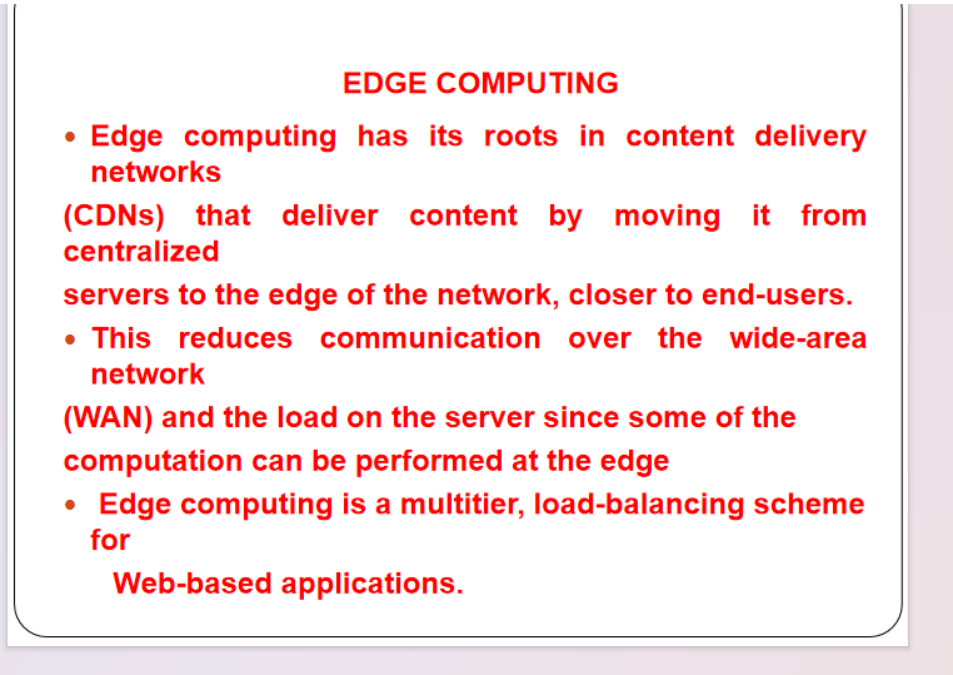
```
print("Style changed successfully.")#DISPLAYING THE SUCCESS MESSAGE
Style changed successfully.
[ ]:
```



BEFORE MANIPULATING



AFTER MANIPULATING THE TEXT



- ADDING THE IMAGE AT THE END AND SAVING IT AS ANOTHER PPT

```
]: from pptx import Presentation
   from pptx.util import Inches

   prs = Presentation(r"C:\Users\KISHORI KAMALI\Downloads\PPT1.pptx")

   last_slide = prs.slides[-1]

   image_path = r"C:\Users\KISHORI KAMALI\Downloads\logo1.png"

   last_slide.shapes.add_picture(
       image_path,
       Inches(1),
       Inches(2),
       width=Inches(3),
       height=Inches(2)
   )#ADDING THE IMAGE

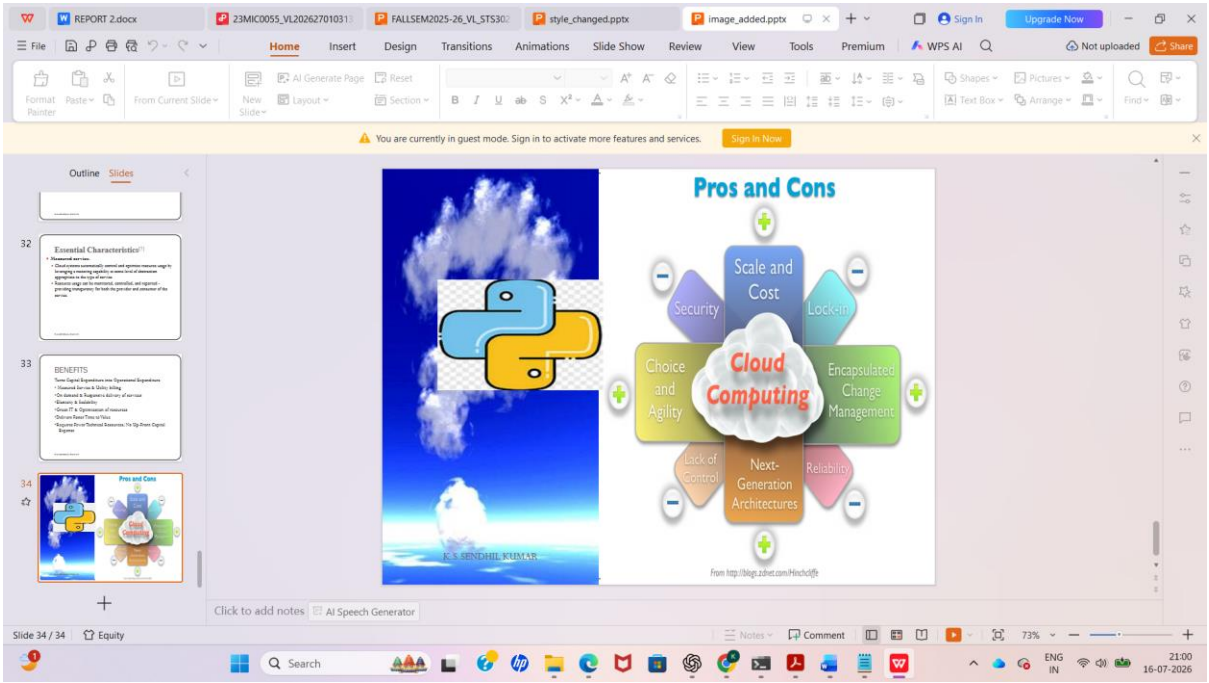
   prs.save("image_added.pptx")# SAVING IT AS ANOTHER PPT

   print("Image added successfully.")#DISPLAYING THE SUCCESS MESSAGE
```

OUTPUT:-

```
Image added successfully.

[ ]:
```



- EXTRACTING THE TEXT FROM THE SLIDE AND DISPLAYING IT

```
2]: from pptx import Presentation

prs = Presentation(r"C:\Users\KISHORI KAMALI\Downloads\PPT1.pptx")

for i, slide in enumerate(prs.slides):#READING THE SLLIDE AND TEXT

    print(f"\nSlide {i+1}")

    for shape in slide.shapes:

        if shape.has_text_frame:
            print(shape.text)#PRINTING THE EXTRACTED TEXT FROM THE SLIDE
```

OUTPUT:-

```
Slide 1
K.S.SENDHIL KUMAR
Associate Professor
SCOPE,VIT University, Vellore.
COMPUTING PARADIGMS

Slide 2
 Edge Computing
 Grid Computing
 Utility Computing
 Distributed Computing
 Cloud Computing

Slide 3
EDGE COMPUTING
Edge computing has its roots in content delivery networks
(CDNs) that deliver content by moving it from centralized
```

- READING THE NOTES FROM THE SLIDES AND DIAPLAYING IT

```
3]: from pptx import Presentation
import pandas as pd

def extract_speaker_notes(pptx_path):#FUNCTION STARTS

    prs = Presentation(pptx_path)#LOADS THE PPT
    notes_log = {}

    for idx, slide in enumerate(prs.slides):#CHECKING EACH SLIDE

        notes_slide = slide.notes_slide
        text_frame = notes_slide.notes_text_frame

        raw_notes = text_frame.text.strip()

        if raw_notes:
            notes_log[f"Slide {idx+1}"] = raw_notes#SLIDE HAS NOTES GET STORED IN THE NOTES_SLIDE ARRAY
        else:
            notes_log[f"Slide {idx+1}"] = "No speaker notes added"#IF NO NOTES IS WRITTEN THEN THIS COMMAND GETS EXECUTED

    df = pd.DataFrame(
        list(notes_log.items()),
        columns=["Slide Location", "Presenter Transcript"]#DESCRIBING THE NOTES
    )

    return df


df = extract_speaker_notes(r"C:\Users\KISHORI KAMALI\Downloads\PPT2.pptx")#EXTRACTING NOTES BY CALLING A FUNCTION AND STORED IT IN VARIABLE
print(df)#PRINTING THE EXTRACTED NOTES
```

```
  Slide Location                          Presenter Transcript
```

OUTPUT:-

	Slide Location	Presenter Transcript
0	Slide 1	“Critical thinking is the most important facto...
1	Slide 2	Critical thinking skills are essential in ever...
2	Slide 3	4. Communication\nCommunication skills are imp...
3	Slide 4	Critical thinking skills are important because...
4	Slide 5	Critical thinking is a domain-general thinking...
5	Slide 6	While you might already possess many of the sk...
6	Slide 7	We know that critical thinking is good for soc...
7	Slide 8	7. Promotes Curiosity\n \nCritical thinkers ar...
8	Slide 9	No speaker notes added

4. GETTING THE AMPLITITUDE AND DOMINANT FREQUENCY FROM THE AUDIO AND PLOTTING THE WAVEFORM

```
import librosa
import numpy as np

# Load audio
audio, sample_rate = librosa.load(
    r"C:\Users\KISHORI KAMALI\Downloads\audio1.wav",
    sr=None
)

# Maximum amplitude
amplitude = np.max(np.abs(audio))

print("Sample Rate:", sample_rate)
print("Maximum Amplitude:", amplitude)
 #FFT
fft = np.fft.fft(audio)

# Frequency bins
frequencies = np.fft.fftfreq(len(audio), d=1/sample_rate)

# Positive frequencies only
positive_freq = frequencies[:len(frequencies)//2]
positive_fft = np.abs(fft[:len(fft)//2])

# Dominant frequency
dominant_frequency = positive_freq[np.argmax(positive_fft)]

print("Dominant Frequency:", dominant_frequency, "Hz")

#plotting the waveform of frequency
plt.figure(figsize=(10,4))
librosa.display.waveshow(audio, sr=sample_rate)

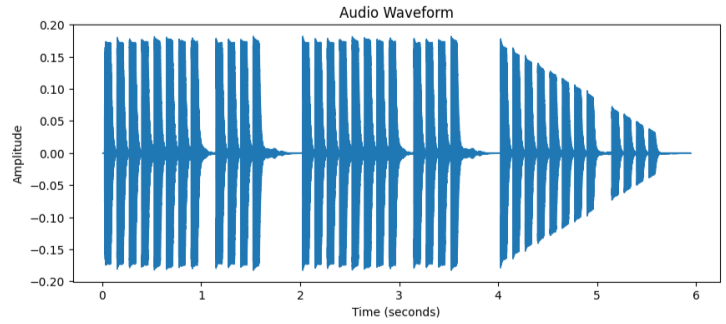
plt.title("Audio Waveform")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")

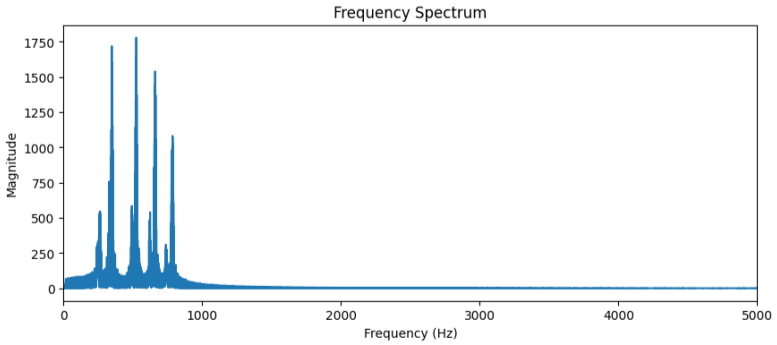
plt.show()
fft = np.fft.fft(audio)
frequencies = np.fft.fftfreq(len(audio), d=1/sample_rate)

plt.figure(figsize=(10,4))
plt.plot(
    frequencies[:len(frequencies)//2],
```

OUTPUT:-

```
Sample Rate: 44100
Maximum Amplitude: 0.1822815
Dominant Frequency: 523.962395171981 Hz
```





BONUS TASK:-

WEB SCRAPING

```
]: import requests
from bs4 import BeautifulSoup
import pandas as pd

# base url
base_url = "https://webscraper.io/test-sites/e-commerce/static/computers/laptops?page={}"

# store extracted data
data = []
total_pages = 20# number of pages to scrape
for page in range(1, total_pages + 1):

    print(f"Scraping Page {page}...")

    url = base_url.format(page)

    response = requests.get(url)

    if response.status_code != 200:
        print("Failed to fetch page:", page)
        continue
    soup = BeautifulSoup(response.text, "html.parser")

    # Find all product cards
    products = soup.find_all("div", class_="thumbnail")
    for product in products:

        name = product.find("a", class_="title").get("title")
        price = product.find("h4", class_="price").text.strip()
        description = product.find("p", class_="description").text.strip()
        reviews = product.find("p", class_="review-count").text.strip()
        rating = len(product.find_all("span", class_="ws-icon-star"))

        data.append({
            "Product Name": name,
            "Price": price,
            "Description": description,
            "Reviews": reviews,
            "Rating": rating
        })
df = pd.DataFrame(data)# converting it into dataframe
print(df)# display data
df.to_csv("products.csv", index=False)# save to csv
print("\nData saved successfully as products.csv")
```

OUTPUT:-

```
Scraping Page 1...
Scraping Page 2...
Scraping Page 3...
Scraping Page 4...
Scraping Page 5...
Scraping Page 6...
Scraping Page 7...
Scraping Page 8...
Scraping Page 9...
Scraping Page 10...
Scraping Page 11...
Scraping Page 12...
Scraping Page 13...
Scraping Page 14...
Scraping Page 15...
Scraping Page 16...
Scraping Page 17...
Scraping Page 18...
Scraping Page 19...
Scraping Page 20...
                            Product Name     Price  \
0                          Packard 255 G2   $416.99
1                           Aspire E1-510   $306.99
2                          ThinkPad T540p  $1178.99
3                                ProBook   $739.99
4                           ThinkPad X240  $1311.99
..                                    ...       ...
112                 Lenovo ThinkPad T470  $1349.23
113        Lenovo ThinkPad Yoga 370 Black  $1362.24
114  Toshiba Portege X20W-D-10V Black/Blue  $1366.32
115     Asus ASUSPRO B9440UA-GV0279R Gray  $1381.13
116          Asus ROG Strix GL702VM-GC146T     $1399

                                       Description     Reviews  Rating
0    15.6", AMD E2-3800 1.3GHz, 4GB, 500GB, Windows...   2 reviews       2
1      15.6", Pentium N3520 2.16GHz, 4GB, 500GB, Linux   2 reviews       3
2     15.6", Core i5-4200M, 4GB, 500GB, Win7 Pro 64bit   2 reviews       1
3      14", Core i5 2.6GHz, 4GB, 500GB, Win7 Pro 64bit   8 reviews       4
4    12.5", Core i5-4300U, 8GB, 240GB SSD, Win7 Pro...  12 reviews       3
..                                                 ...         ...     ...
112  Lenovo ThinkPad T470, 14" FHD IPS, Core i5-720...   5 reviews       1
113  Lenovo ThinkPad Yoga 370 Black, 13.3" FHD IPS ...  12 reviews       2
114  Toshiba Portege X20W-D-10V Black/Blue, 12.5" F...  11 reviews       1
115  Asus ASUSPRO B9440UA-GV0279R Gray, 14" FHD, Co...   4 reviews       1
116  Asus ROG Strix GL702VM-GC146T, 17.3" FHD, Core...  10 reviews       3

[117 rows x 5 columns]

Data saved successfully as products.csv
```